

GD2 A2

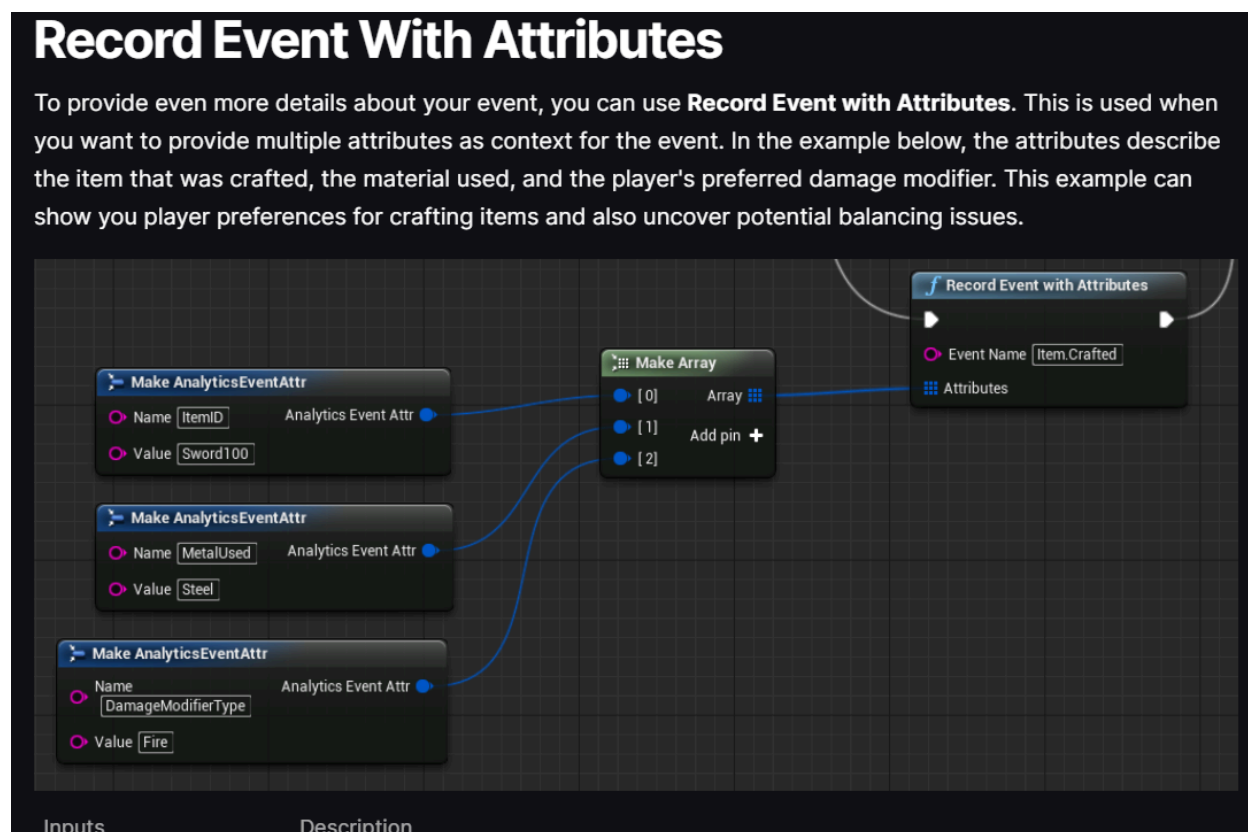
Cole Dorman

To start on my creation of the telemetry system I began by firstly researching online to see what unreal engine 5 had built in to handle telemetry, I found the following:

[Unreal Engine Blueprint Analytics Plugin](#)

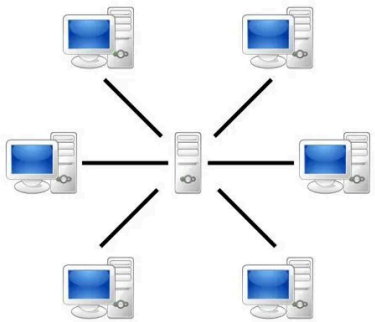
[Unreal Insights](#)

The Blueprint Analytics plugin turned out to not have the functionality and flexibility I needed, and Unreal Insights was primarily for performance, so it was not directly useful. The image below is an example of why I had thought the Analytics Plugin would be what I had needed.



It however lacked many key features such as being able to actually send these events to a telemetry server for handling.

Testing



Immediately since I needed to do LAN based networking to send data from clients to the server, I had run some smaller scale testing by using a PowerShell script to simulate fake data, using a Sin wave, and then I had used a Python http server to receive the data and then store in it a file. The tests worked flawless and showed no data loss over LAN even when under pressure of large amount of requests per second (> 250). Once these succeeded I had a proof of concept I got to work creating a framework for how I would structure my telemetry system as a C++ unreal engine plugin.

My client testing script (The Sender)

```
# PowerShell client script to send test telemetry data
param(
    #laptop vpn ip = "10.243.11.239"
    #desktop ip = "10.20.5.27"
    [string]$ServerIP = "10.20.5.27",
    [int]$Port = 8080,
    [string]$MachineID = $env:COMPUTERNAME,
    [int]$DurationSeconds = 60,
    [int]$IntervalMs = 100
)

$url = "http://$($ServerIP):$($Port)/telemetry"
$startTime = Get-Date
$counter = 0
```

This is the configuration I used for my running of the script itself. This was tested between 3 different computers in the classroom.

```
# Simulate game telemetry data via sin wave
$telemetryData = @{
    machine_id = $MachineID
    frame = $counter
    player_pos = @{
        x = [math]::Sin($counter * 0.1) * 100
        y = [math]::Cos($counter * 0.1) * 100
        z = 0
    }
    fps = Get-Random -Minimum 55 -Maximum 65
    memory_mb = Get-Random -Minimum 2000 -Maximum 3000
    active_enemies = Get-Random -Minimum 0 -Maximum 10
} | ConvertTo-Json -Compress
```

Here we are faking data by just using a sin wave to simulate a change over time

```
# Send POST request
try {
    $response = Invoke-WebRequest -Uri $url -Method POST
    -Body $telemetryData -ContentType "application/json"
    -UseBasicParsing -TimeoutSec 2
}
```

This is where we actually sent the request via a POST request

Telemetry server script (The Receiver)

```
DATA_FILE = Path("data") / "telemetry.jsonl"
_write_lock = threading.Lock()
_event_count = 0
```

These lines set up where telemetry data will be stored and prepare for safe handling when multiple devices send data at once.

```

def do_POST(self):
    if self.path == '/telemetry':
        try:
            # Read request body
            content_length = int(self.headers['Content-Length'])
            body = self.rfile.read(content_length)
            data = json.loads(body.decode('utf-8'))

            # Add server-side timestamp for verification
            data['server_timestamp'] = datetime.now().isoformat()

            # Ensure data directory exists
            self.DATA_FILE.parent.mkdir(parents=True,
                                         exist_ok=True)

            # Thread-safe write to single JSONL file (one JSON
            # object per line)
            with self._write_lock:
                with open(self.DATA_FILE, 'a') as f:
                    f.write(json.dumps(data) + '\n')
                    f.flush() # Ensure immediate write to disk

            # Increment event counter
            TelemetryHandler._event_count += 1

```

This function handles incoming telemetry data. When a client sends data to /telemetry, it adds a server timestamp and safely stores the information in the log file, even when multiple devices send data simultaneously.

```
def run_server(port=8080):
    server_address = ('0.0.0.0', port)
    httpd = HTTPServer(server_address, TelemetryHandler)

    print("=" * 60)
    print("UE5 TELEMETRY SERVER")
    print("=" * 60)
    print(f"Server running on port {port}")
    print(f>Data file: {TelemetryHandler.DATA_FILE.absolute()}")
    print("Listening for events from UE5 clients...")
    print("Press Ctrl+C to stop\n")

    try:
        httpd.serve_forever()
    except KeyboardInterrupt:
        print(f"\n\nServer stopped. Total events received:
        {TelemetryHandler._event_count}")
        print(f>Data saved to: {TelemetryHandler.DATA_FILE.absolute()}
        ")
```

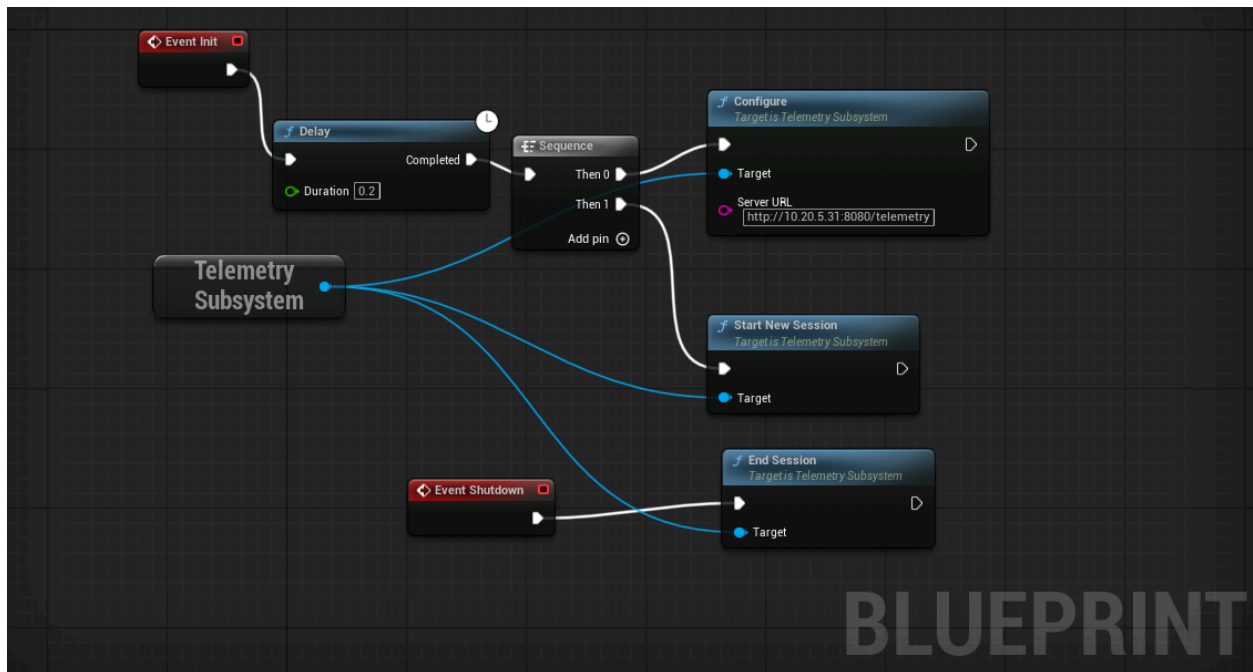
This function starts the telemetry server, showing which port it's listening on. When you press Ctrl+C to stop it, it reports how many events were received and where the data was saved.

Refinement

After realizing the limitations, I had consulted Ivan for direction and had taken a look at a potential C++ implementation of my own telemetry system. Since I had tested using HTTP to send json over LAN to a host from a client I started by looking into unreal engine's modules in C++. After some research I had found Json module and its helper module; JsonUtilities. Being able to utilize the reflection based serialization that unreal engine already has native, kept my code clean and reduces overhead.

Implementation

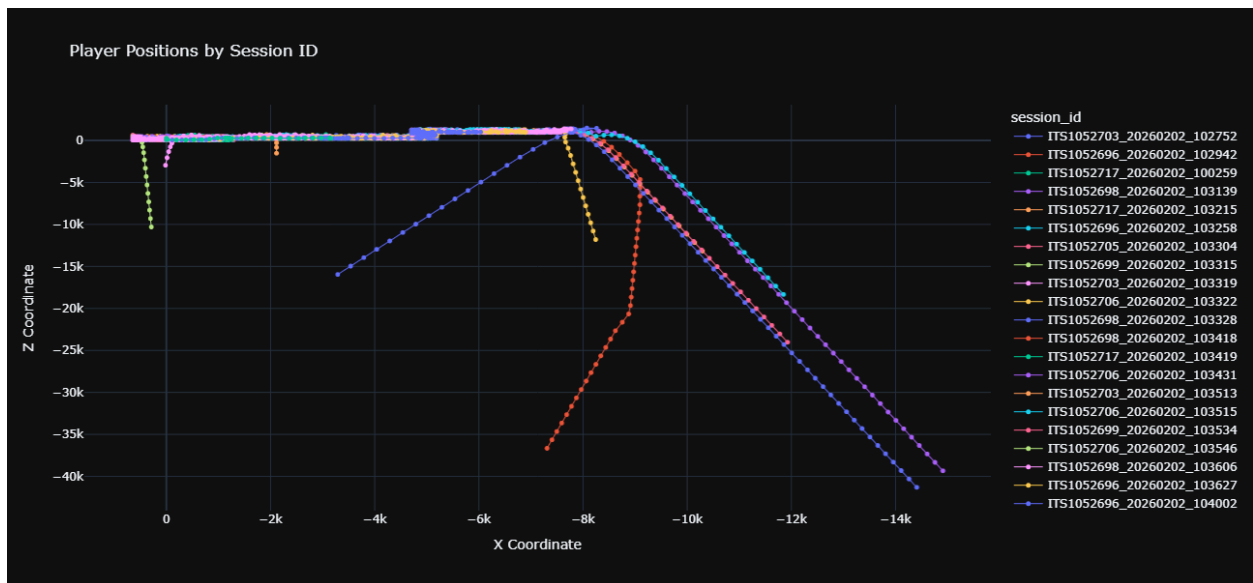
Once the C++ plugin was created, and I had blueprint callable methods setup, this was the exact blueprint implementation I had set up for the first playable.



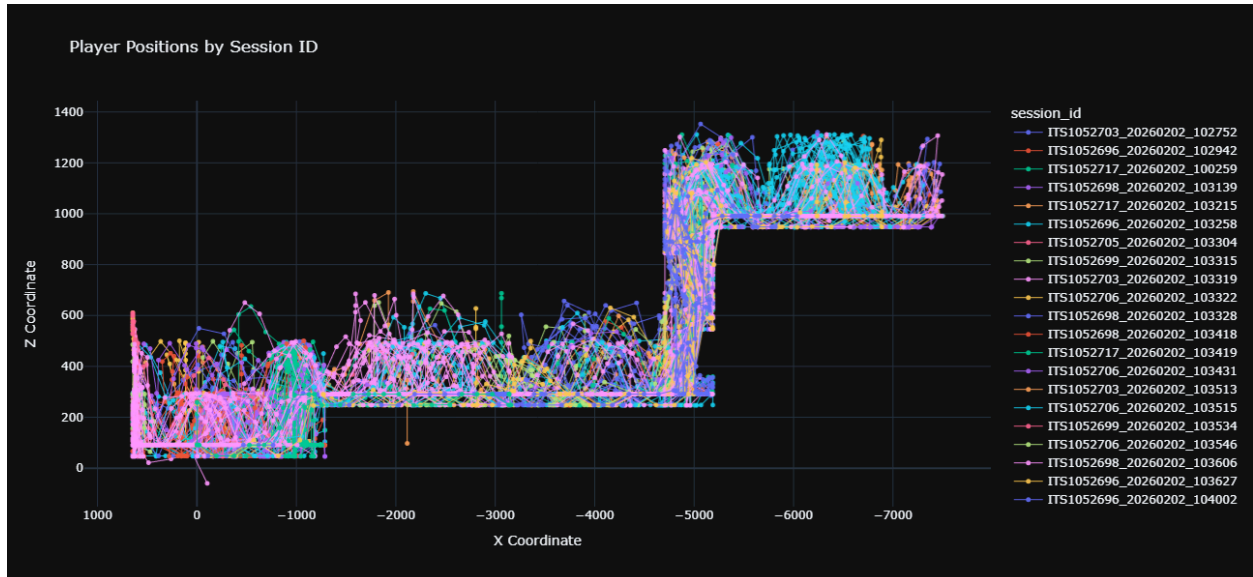
This is the EventGraph of the GamelInstance, which is where I handle the setup of the Telemetry Subsystem I had created.

Once I had initialized and set up the telemetry system in the game instance, it was utilized from within the Player_BP's EventGraph, via a looping timer.

This is what it would look like if plotted without constraining to the level bounds and cleaning it:

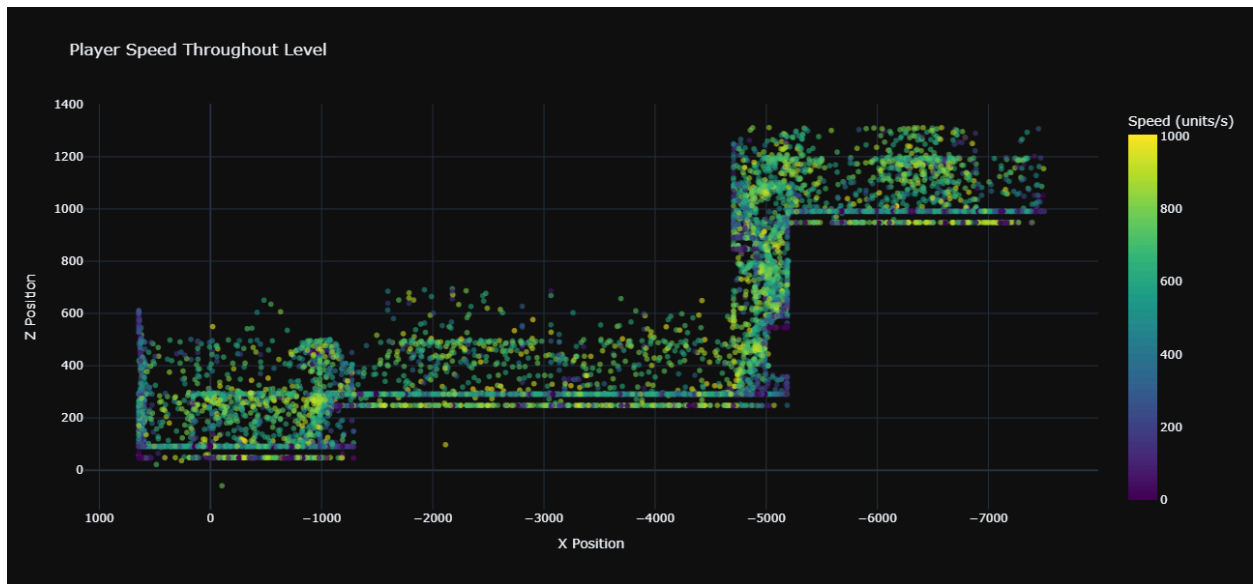


This chart below shows the cleaned and organized data:



*Lines are drawn to show the path each player took, each session_id is a unique player's path.
(Some have similar colors)*

Here is the same data except we are visualizing velocity, in order to identify areas where players tend to dwell at.



Faster is more yellow, while slow is dark purple.